

JAVA INTERVIEW PREPARATION

CHAPTER 73

DYNAMIC PROGRAMMING,
STATE MODELING, AND RECONSTRUCTION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Dynamic Programming: State Modeling, Transitions, and Reconstruction

Senior / Lead Java Developer Reading Chapter

Dynamic programming, or DP, solves a problem by identifying repeated subproblems, defining the information that uniquely describes each subproblem, and storing answers so each state is solved once. It is not a bag of table-filling tricks. The main intellectual work is state design: what must be remembered about the past so the future can be solved independently?

Senior interviews use DP to test whether a candidate can move from brute-force choices to a recurrence, justify evaluation order, control memory, reconstruct an actual solution, and recognize when the state space is too large for production. A correct recurrence with an incomplete state is wrong; a complete state with exponential dimensions may be impractical.

From repeated recursion to stored states

The recursive Fibonacci definition demonstrates overlap:

```
fib(5)
├─ fib(4)
│  └─ fib(3)
│     └─ fib(2)
├─ fib(3)      repeated
│  └─ fib(2)    repeated
└─ fib(1)
```

Naive recursion recomputes the same `fib(n)` states, creating exponential calls. Memoization stores each result.

```
static long fibonacci(int n, long[] memo) {
    if (n < 0) throw new IllegalArgumentException("n");
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = Math.addExact(
        fibonacci(n - 1, memo),
        fibonacci(n - 2, memo));
}
```

This example uses zero as "not computed" only because every stored state for $n \geq 2$ is positive. General memoization needs a separate presence flag, nullable wrapper, or map. Sentinel values that overlap real answers create silent bugs.

The state is n . The recurrence is $F(n) = F(n - 1) + F(n - 2)$. Base states are 0 and 1. The dependency graph is acyclic because every transition reduces n .

The four-part DP design

A disciplined DP explanation states:

1. State: what does `dp[...]` mean? 2. Transition: how does this state depend on smaller states? 3. Base cases: which states are known directly? 4. Evaluation order: when a state is computed, are its dependencies ready?

Then add answer location, complexity, numeric range, and reconstruction.

```
problem choices
  |
  v
identify repeated future situations
  |
  v
define minimal complete state
  |
  v
recurrence + bases + evaluation order
```

Starting with a table shape before defining meaning is a common source of accidental solutions that cannot be explained or modified.

Top-down memoization

Top-down DP preserves the natural recursive structure. It evaluates only states reachable from the requested answer and can be easy to derive from brute force.

```
static int minCost(int index, int[] cost, Integer[] memo) {
    if (index >= cost.length) return 0;
    if (memo[index] != null) return memo[index];
    int one = minCost(index + 1, cost, memo);
    int two = minCost(index + 2, cost, memo);
    return memo[index] = cost[index] + Math.min(one, two);
}
```

Advantages include demand-driven state evaluation and close correspondence with the recurrence. Costs include recursion depth, function-call overhead, object wrappers if null is used, and less predictable memory access. Cyclic state dependencies require cycle handling; memoization alone does not make a cyclic recurrence well-defined.

For concurrent requests, memo tables are normally request-local. A shared cache is safe only when inputs are immutable and fully represented in the key, lifetime is bounded, and eviction/observability are designed.

Bottom-up tabulation

Bottom-up DP evaluates states in a dependency order without recursive calls.

```
static long fibonacci(int n) {
    if (n < 0) throw new IllegalArgumentException("n");
    if (n <= 1) return n;
    long previous = 0;
    long current = 1;
    for (int i = 2; i <= n; i++) {
        long next = Math.addExact(previous, current);
        previous = current;
        current = next;
    }
    return current;
}
```

The full recurrence needs only the previous two states, so memory compresses from $O(n)$ to $O(1)$. Compression is safe because older values will never again be dependencies. If solution reconstruction needs the full decision history, keep parent information or recompute carefully.

Bottom-up code may compute states that top-down search would never reach, but it avoids stack limits and usually has strong locality.

Grid paths and boundary modeling

For a grid with blocked cells, define $dp[r][c]$ as the number of ways to reach cell (r, c) from the origin using allowed moves.

```
S . .  
. X .  
. . T
```

```
dp[r][c] = dp[r-1][c] + dp[r][c-1]  
unless the cell is blocked
```

```
static long countPaths(boolean[][] blocked) {  
    if (blocked.length == 0 || blocked[0].length == 0) return 0;  
    int columns = blocked[0].length;  
    long[] dp = new long[columns];  
    dp[0] = blocked[0][0] ? 0 : 1;  
  
    for (boolean[] row : blocked) {  
        if (row.length != columns) throw new IllegalArgumentException("ragged");  
        for (int c = 0; c < columns; c++) {  
            if (row[c]) {  
                dp[c] = 0;  
            } else if (c > 0) {  
                dp[c] = Math.addExact(dp[c], dp[c - 1]);  
            }  
        }  
    }  
    return dp[columns - 1];  
}
```

Before updating, $dp[c]$ is the value from the previous row; $dp[c - 1]$ is the newly updated value for the current row. This dependency explains why iteration runs left to right. Reversing the direction would use the wrong logical row.

Path counts grow rapidly. The domain may require modulo arithmetic, `BigInteger`, saturation, or an explicit overflow failure. Choose deliberately.

0/1 knapsack and loop direction

In 0/1 knapsack, each item may be chosen at most once. Define $dp[c]$ as the best value achievable with capacity at most c after processing a prefix of items.

```
static long knapsack(int[] weight, int[] value, int capacity) {  
    if (weight.length != value.length || capacity < 0) {  
        throw new IllegalArgumentException();  
    }  
    long[] dp = new long[capacity + 1];  
  
    for (int i = 0; i < weight.length; i++) {  
        if (weight[i] <= 0) throw new IllegalArgumentException("weight");  
        for (int c = capacity; c >= weight[i]; c--) {  
            dp[c] = Math.max(dp[c],  
                             dp[c - weight[i]] + value[i]);  
        }  
    }  
    return dp[capacity];  
}
```

Capacity iterates downward so $dp[c - \text{weight}]$ still belongs to the previous item-prefix. Iterating upward would reuse the current item multiple times, solving unbounded knapsack instead.

0/1 item: capacity right -> left
unbounded item: capacity left -> right

This is a powerful interview example because an apparently minor loop direction encodes the problem's reuse semantics.

Coin change has multiple problem statements

"Coin change" may ask for minimum coins, number of combinations, number of ordered sequences, or feasibility. They require different recurrences.

For unordered combinations with unlimited coins, iterate coins outside and amounts upward:

```
static long countCombinations(int[] coins, int amount) {
    long[] ways = new long[amount + 1];
    ways[0] = 1;
    for (int coin : coins) {
        if (coin <= 0) throw new IllegalArgumentException("coin");
        for (int sum = coin; sum <= amount; sum++) {
            ways[sum] = Math.addExact(ways[sum], ways[sum - coin]);
        }
    }
    return ways[amount];
}
```

Reversing loop nesting counts ordered sequences because each amount considers every possible final coin. A strong answer defines whether `[1,2]` and `[2,1]` are the same solution before writing code.

Minimum-coin DP initializes unreachable states to a safe sentinel and minimizes `1 + dp[amount - coin]`. Never add one to `Integer.MAX_VALUE`; guard the sentinel to avoid overflow.

Longest common subsequence

For strings or sequences A and B, define `dp[i][j]` as the LCS length of suffixes starting at `i` and `j`, or prefixes ending before them. Choose one convention and keep bases consistent.

```
A = A B C D
B = A C E D

one LCS = A C D
```

For prefix lengths:

```
if A[i-1] == B[j-1]: dp[i][j] = 1 + dp[i-1][j-1]
otherwise:          dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Time and space are $O(mn)$. Length alone can be compressed to two rows, but reconstructing the subsequence typically keeps the table or uses a more advanced divide-and-conquer technique. During reconstruction, equal alternatives may produce different valid LCS results; deterministic tie-breaking should be explicit if output stability matters.

For Java strings, index semantics matter. `charAt` compares UTF-16 code units. A code-point sequence can be created first when that is the required unit; user-perceived graphemes require a higher-level segmentation policy.

Edit distance

Levenshtein distance defines `dp[i][j]` as the minimum edits converting the first `i` units of A into the first `j` units of B.

```
delete:    dp[i-1][j] + 1
insert:    dp[i][j-1] + 1
replace:    dp[i-1][j-1] + cost
```

The first row converts empty A to prefixes of B by insertion; the first column uses deletions. Weighted edit distance can assign domain-specific costs. Transposition requires a different recurrence such as Damerau-Levenshtein.

Computing a full matrix is $O(mn)$, which is dangerous for unbounded strings. If only distances below threshold k matter, banded DP explores a diagonal band and can stop when no state remains competitive. Search products often combine candidate generation with bounded edit distance instead of comparing every dictionary term.

Longest increasing subsequence

A straightforward DP defines $dp[i]$ as the longest increasing subsequence ending at i :

```
dp[i] = 1 + max(dp[j]) for j < i and a[j] < a[i]
```

This is $O(n^2)$. A more efficient $O(n \log n)$ method maintains $tails[length]$, the smallest possible tail value for an increasing subsequence of that length, and uses lower-bound search.

```
values: 3, 5, 2, 6, 4
tails progression:
[3]
[3,5]
[2,5]
[2,5,6]
[2,4,6]
```

The tails array is not necessarily an actual subsequence. It summarizes possibilities. Reconstructing an LIS requires predecessor and position arrays. Strictly increasing versus nondecreasing changes lower-bound to upper-bound behavior.

Interval dynamic programming

Some states represent an interval rather than a prefix. Matrix-chain multiplication, palindrome partitioning, and burst-balloon variants choose a split or final operation inside $[left, right]$.

```
dp[left][right]
= best over split k of
  dp[left][k] + dp[k+1][right] + combineCost
```

Evaluation proceeds by increasing interval length so smaller intervals are ready first.

```
length 1 states
length 2 states
length 3 states
...
full interval
```

Typical time is $O(n^3)$ and space $O(n^2)$. That is acceptable only for moderate n . The recurrence may be mathematically elegant but operationally impossible for n in the tens of thousands.

Bitmask DP

When n is small, a bit mask can represent a subset of visited items. Traveling-salesperson-style state may be $dp[mask][last]$: minimum cost to visit exactly $mask$ and finish at $last$.

The state count is $O(2^n * n)$, with transitions often adding another factor n . This is far better than $n!$ brute force but still exponential. For $n = 25$, $2^n * n$ is already hundreds of millions of states. Estimate bytes before allocating an array.

Use `1L << n` rather than `1 << n` when a long mask is intended, validate shift widths, and understand that Java shifts mask the shift distance. For larger sets, `BitSet` may represent state but is expensive as a memo key unless copied and hashed carefully.

Pseudo-polynomial complexity

Knapsack with $O(n * \text{capacity})$ time is polynomial in the numeric capacity value, but not in the number of bits required to encode that value. A capacity of one billion arrives in about 30 bits yet makes the table infeasible. This is called pseudo-polynomial complexity.

The distinction matters in architecture reviews. Input record count may be small while a monetary, time, or capacity dimension is enormous. Scaling or rounding values can create an approximation scheme, but the resulting error bound and business meaning must be explicit. Sparse reachable sums can help when few totals occur, though worst-case state count may still be large.

State compression and update hazards

Memory can be compressed when dependencies occupy only previous rows, columns, or a bounded window. The update direction must prevent overwriting a value still needed later in the same iteration.

Before compressing, draw dependencies:

```
new dp[c] depends on:
  old dp[c]      (above)
  new dp[c-1]    (left)

therefore iterate left to right
```

Compression can make reconstruction harder and code less obvious. Do it after correctness, and preserve a comment stating the logical old/new meaning of each cell.

Sparse DP uses maps when only a small portion of a theoretical state space is reachable. It saves memory but adds hashing, allocation, and nondeterministic iteration. Dominance pruning can discard a state that is no better than another under every future criterion, but the dominance proof must be valid.

Map keys should be compact immutable values. A record can work for a few integer dimensions; mutable arrays or `BitSet` instances must be copied before use as keys. Canonicalize symmetric states when possible so equivalent arrangements share one memo entry. Otherwise the algorithm may technically memoize while still exploring the same logical state under many representations.

Memo lifetime is a production concern. A cache scoped to one input can be released when the request ends. A static cache keyed only by state indices is wrong across different input arrays, dictionaries, or cost models. A fully keyed shared cache can still grow without bound and retain large inputs. Prefer request-local tables unless cross-request reuse is proven and governed.

Reconstructing the decision

Many interview solutions return only an optimum value even though the domain needs the chosen items, path, or edits. Reconstruction options include:

- store the selected transition or parent per state;
- compare neighboring table values while walking backward;

- rerun decisions using the completed value table;
- use divide-and-conquer reconstruction to reduce memory.

Tie behavior matters. Two optimal solutions may exist. Define whether any solution is acceptable, lexicographically smallest is required, or stable business priority decides.

```
value table answers: "how good?"
parent/choice data answers: "which decisions produced it?"
```

Production applications and limits

DP appears in sequence comparison, resource allocation, pricing, batching, routing, parsing, capacity planning, and probabilistic models. Production implementations must bound state count, numeric growth, and input dimensions. $O(mn)$ is harmless for 100 by 100 and catastrophic for 100,000 by 100,000.

Estimate memory explicitly. A `long[10_000][10_000]` contains about 800 MB of primitive payload before row-object overhead. Boxed maps can require several times more. Prefer flat primitive arrays when dense, reuse work buffers where safe, and expose metrics for state count, cache hit rate, allocation, and cutoff reason.

A flat array can improve locality and reduce row objects: map `(row, column)` to `row * columns + column`, using `long` to validate multiplication before allocating. Reusing buffers across requests requires strict ownership; pooling a huge array can reduce allocation while retaining an undesirable memory high-water mark.

Approximation, heuristics, or decomposition may be required when exact DP exceeds the budget. Report whether the answer is exact, bounded, or best-effort. Do not turn an out-of-memory failure into the implicit feasibility test.

Testing dynamic programs

Start with a brute-force solver for tiny inputs and compare randomized cases. Test empty state, one element, impossible states, all choices equal, negative values where allowed, duplicate items, maximum capacities, overflow boundaries, and multiple optimal solutions.

Useful properties include:

- top-down and bottom-up implementations agree;
- compressed and full-table results agree;
- reconstructed choices are feasible and produce the reported value;
- adding capacity cannot reduce an "at most capacity" optimum;
- pruning or sparse-state elimination matches the unoptimized recurrence on small input;
- loop direction matches 0/1 versus reusable-item semantics.

Common senior-level traps

- Starting with a table before defining state meaning.
- Using an incomplete memo key.
- Choosing bases inconsistent with prefix/suffix conventions.
- Iterating compressed state in the wrong direction.
- Confusing combinations with ordered sequences in coin change.
- Adding to an infinity sentinel and overflowing.

- Returning only optimum value when the decision is required.
- Claiming polynomial complexity while one dimension is a numeric capacity with huge magnitude.
- Treating $O(n^2)$ memory as acceptable without estimating bytes.
- Applying DP when subproblems do not overlap.
- Using `char` while claiming general Unicode-character semantics.

A strong interview answer

I first derive a brute-force choice recurrence, then identify which information makes the remaining future independent of the path. That becomes the minimal complete state. I state the recurrence, base cases, evaluation order, and answer location before coding. Top-down memoization is demand-driven but uses recursion; bottom-up tabulation controls order and often compresses memory. I justify update direction, numeric type, and reconstruction. Finally, I calculate state count in real units and explain whether exact DP fits production memory and latency or needs sparse states, pruning, approximation, or a different formulation.

Chapter summary

Dynamic programming stores answers to overlapping subproblems defined by a complete state. Recurrence, base cases, and evaluation order form one correctness contract. Top-down and bottom-up approaches compute the same dependency DAG differently. Loop direction encodes reuse semantics, compression follows dependency lifetime, and reconstruction requires decision evidence. Production use demands explicit state-space and memory arithmetic rather than relying on polynomial notation alone.

Revision Notes

- DP needs overlapping subproblems and reusable state answers.
- Define state meaning before table shape.
- State, transition, bases, and evaluation order form the core design.
- Memoization computes reachable states but retains recursion depth.
- Tabulation computes in dependency order and often improves locality.
- Sentinel values must not overlap valid results.
- Compress memory only after drawing dependencies.
- Grid one-row DP direction follows old-above and new-left dependencies.
- 0/1 knapsack iterates capacity downward; unbounded reuse iterates upward.
- Coin loop order distinguishes combinations from sequences.
- Guard arithmetic around infinity sentinels and large counts.
- LCS and edit distance are commonly $O(mn)$ time and space before compression.
- LIS can improve from $O(n^2)$ to $O(n \log n)$ with tails and binary search.
- Interval DP usually evaluates increasing interval lengths.
- Bitmask DP is exponential even when better than brute-force permutations.
- Reconstruction needs parents, decisions, or a value-table backtrack.
- Estimate real memory and define exact versus approximate termination.

Likely interview question: How do you identify a DP problem?

Start from recursive choices. If multiple paths reach the same future situation and the best answer from that situation can be reused, the problem has overlapping subproblems and optimal substructure. Define the smallest complete description of that future situation.

Likely interview question: Top-down versus bottom-up?

Top-down mirrors the recurrence and computes only reachable states but uses recursion and memo lookup. Bottom-up requires a valid dependency order, avoids stack depth, and often offers better locality and compression. Choose based on reachability, depth, and clarity.

Likely interview question: Why does loop direction matter in knapsack?

In one-dimensional compression, descending capacity reads values from the previous item stage, so an item is used once. Ascending capacity reads values updated during the same item stage, permitting repeated use. The loop direction encodes the reuse contract.

Likely interview question: How do you make DP production-safe?

Bound dimensions, estimate state bytes, use primitive or sparse storage appropriately, detect numeric overflow, support cancellation or cutoffs, reconstruct only when needed, compare against a small oracle, and state whether any fallback is exact or approximate.